PAYPAL AI HACKATHON

The frontend, as it stands

What is built in the React app, and what Person 3 still has to ship before 11 November.

OWNER	STACK	SCREENS	SUBMIT
Person 3	React 18 + Vite	7 app screens	11 Nov 2026

Source: frontend/ source, docs/, and the PayScore team work split.

Amounts in the product are INR. This note uses Rs so the type stays clear.

1. At a glance

The frontend is a React application in **frontend/**. It is the credit-copilot interface for the PayPal AI hackathon: a seller links a PayPal sandbox account, or tours the product on synthetic data, then walks income, an explainable PayScore, an offer check, a chat copilot, repayment, and a widget cockpit.

Person 3 (India, PayPal access) owns the agent and every UI screen. The team split keeps the Pakistan-based teammate on Python and mock data. Final testing, the demo video, and submission are led by Person 4, with Persons 2 and 3 fixing what breaks. The aim is to submit on **11 November 2026**, one day before the 12 November, 12:00 pm PT deadline.

Where the work sits today

Area	What you can open now	Status
Landing and sign-in	PayPal login, or a demo tour when the sandbox is not configured	Built
App shell	Sidebar, mobile drawer, light and dark theme, live score in the top bar	Built
Income	12-month profile, area chart, payment rhythm, AG Grid transaction feed	Built
PayScore	300–850 gauge, risk band, SHAP factors, eligible changes, disbursement limit	Built
Offer guard	Manual terms or a PDF upload, cost comparison, KFS checklist, directory result	Built
AI copilot	Chat against the Python agent, with an approval card before a sandbox payout	Partial
Repayment	Fixed EMI, revenue share, and a one-off PayPal checkout	Built
AG Studio cockpit	Local widget board. Not a custom AG Studio agent	Partial
PayPal MCP copilot	Planned for week 2. The chat does not call the MCP server	Planned
Bryntum calendar	Only if the team is ahead in week 3. No calendar dependency is installed	If ahead

Built means the screen calls a real API. Partial means the screen is usable, and the planned version is still ahead.

2. How the frontend is built

Run it with **npm install** and **npm run dev** inside **frontend/**. Vite serves **http://localhost:5173** and proxies API calls to the FastAPI app on port 8000. Set **PAYSCORE_FRONTEND_URL=http://localhost:5173** on the API so PayPal OAuth returns to **/auth/callback**. Leave **VITE_PAYSCORE_API_URL** empty in local development so requests stay same-origin. Set it only when the UI is hosted apart from the API.

Piece	Choice in package.json
UI	React 18 and TypeScript
Build	Vite 8, route-level code splitting. Lint with oxlint. Production build is tsc -b and vite build.
Routing	React Router 7
Styling	Tailwind CSS 3, Plus Jakarta Sans, PayPal blue #0070BA and gold #F5B942
Motion	Framer Motion. Reduced motion follows the system setting
Charts	Recharts area chart on the income page
Grid	AG Grid Community 32, themed for the income transaction feed

Routes

Every authenticated screen sits under **/app** inside **AppShell**. A missing session sends the visitor back to the landing page. Unknown paths inside the app return to the overview.

Path	Screen	File
/	Landing	pages/Landing.tsx
/auth/callback	PayPal return	pages/AuthCallback.tsx
/app	Overview	pages/Overview.tsx
/app/income	Income profile	pages/Income.tsx
/app/score	PayScore	pages/Score.tsx
/app/offer-guard	Offer guard	pages/OfferGuard.tsx
/app/copilot	AI copilot	pages/Copilot.tsx
/app/repay	Repayment	pages/Repayment.tsx
/app/cockpit	AG Studio cockpit	pages/Cockpit.tsx

pages/Placeholder.tsx is an unused skeleton that says a screen will be wired later. No route renders it.

Shell and session

The shell is a 256px sidebar on large screens and a slide-in drawer on small screens. The top bar shows the wordmark, a live PayScore pill, a demo or PayPal-linked chip, the theme toggle, and sign out. The live pill polls **GET /score/live** every 8 seconds for applicant **PSC-PRIYA-001** and pulses when the number changes.

The session lives in **sessionStorage** under **paypilot.session**. **Log in with PayPal** checks **GET /paypal/status** and, when the sandbox is configured, sends the browser to **GET /paypal/auth/login**. A successful return with **linked=true** opens the app as a linked session. An error on the callback offers a return to the landing page or a continue on demo data. **Tour the cockpit** opens the same app with **demo: true** and persona **priya**, without a PayPal account. Sign out clears the session.

3. What each screen does

Landing

A full-viewport page with an ambient background, the sandbox badge, and a light/dark toggle. The headline is the product claim: banks reject freelancers who have no credit score, and PayPal history can stand in for one. Three steps describe linking a sandbox account, reading 12 months of cashflow, and seeing a 300–850 score that is labelled as a demo, not a bank decision. Floating cards sit beside the copy. If sandbox credentials are missing, the PayPal button stays on the page and a notice points people to the tour.

Overview

Loads **GET /income-profile** and **GET /demo/persona/{name}**. The session persona is always Priya today. Four metric cards show 12-month income, distinct clients, refund rate, and the longest payment gap. A workspace panel links to income, PayScore, and offer guard. A second card is the 3-minute demo path: income, score, offer guard, copilot, and repayment. Visited steps are stored in **paypilot.demo_progress**. A demo session shows a notice that the data is synthetic and that no real credit decision or money movement happens here.

Income

The profile comes from **GET /income-profile?persona=priya**. When the API source is **paypal**, the page says the linked profile is aggregates only. The transaction grid does not come from that endpoint. It comes from the synthetic persona fixture **GET /demo/persona/priya**, field **recent_transactions**.

• Count-up cards for 12-month income, client count, refund rate, and dispute rate.

• A Recharts area chart of monthly_income_12m, drawn over about 1.2 seconds.

• Payment rhythm: monthly direction, longest income gap, and active buyer count.

• An AG Grid with sortable, filterable columns for date, type, amount, category, and status, plus custom cell renderers and a PayPal-blue header theme.

PayScore

Run PayScore posts the Priya **model_input** to **POST /score** and stores the response in **paypilot.score**. The gauge is a ring from 300 to 850 with a needle sweep. The risk band is coloured green, amber, orange, or rose for low, moderate, elevated, and high.

• When explanation.status is available, the factor cards are TreeSHAP log-odds: factors that lower estimated risk and factors that raise it. The copy says these are attributions, not causes.

• A “to move up” line comes back on the score. See eligible changes calls POST /counterfactuals and lists each suggestion with an estimated score and the score difference.

• A disbursement card shows eligible or not eligible, and the approved limit in INR when the score clears the server threshold.

• The page is locked to the Priya fixture. There is no control to score Arjun or Rohan.

Offer guard

The form accepts a lender name, advertised APR (0–100), processing fee (0–50), principal (1 to 1,000,000,000 INR), and term (1–360 months). Leaving the file input empty posts those fields to **POST /offer-guard**. Choosing a PDF posts it to **POST /offer-guard/upload** and ignores the manual numbers. Anything that is not a PDF is rejected in the browser. The result is saved as **paypilot.offer** so the cockpit can read it.

• The result shows advertised APR against the estimated real annualised cost, and highlights a red flag when the server sets red_flag.

• A KFS checklist renders item by item. A directory line shows the RBI match status. The page states that a miss is not a confirmed violation, and that the check stays unverified until a retrieved list is loaded.

• Extraction notes can include OCR status and LLM status. Both are optional on the server and off in a normal install.

AI copilot

The chat posts to **POST /copilot/chat** with applicant **PSC-PRIYA-001**. On first open it waits about 0.8 seconds and sends “Can I afford Rs 50k?”. Suggested questions cover income trend, PayScore comparison, a loan offer, and EMI. Command-K or Control-K focuses the input. Replies show the tool name as a chip and, when the server returns one, the analysis kind and status.

If the server sets **approval.required**, an approval card appears with the action, the amount, and Approve / Decline. Decline adds a local message and calls nothing. Approve reads the stored score, refuses to continue if PayScore has not been run, then calls **POST /disburse** for priya@example.test and saves **paypilot.payout_batch_id**. A link then points at the repayment page to track that batch.

This copilot is the Python agent in payscore/copilot_agent.py. It is not yet wired to the PayPal MCP server. That wiring is the remaining week-2 item for Person 3.

Repayment

Three modes share one page. The ledger polls **GET /repayment/status** every 15 seconds. A payout batch id, from the copilot or typed in, polls **GET /disburse/{batch_id}** every 5 seconds and plays a success animation when the batch reaches SUCCESS or COMPLETED. The page also polls the live score and shows a pulse when it changes.

Mode	What the screen sends	What comes back on screen
Fixed EMI	POST /repayment/subscription with the INR amount and the number of instalments	A schedule of due dates, INR amounts, and the PayPal USD amount
Revenue share	POST /repayment/revenue-share. The fields show 5% and DEMO-PRIYA, and the click handler always sends those two values.	Whatever schedule the server returns. Typing a different percentage does not change the request.
One-off	POST /repayment/order, then the PayPal JS SDK v6 session from GET /paypal/public-config	A Pay with PayPal button. After approval the page captures with POST /repayment/order/{id}/capture and can burst confetti

Subscriptions and orders are presented in INR. The API converts outbound PayPal amounts to USD with a demo rate. The schedule shows both.

AG Studio cockpit

The sidebar label is AG Studio. The page is a local React board, not an AG Grid Studio custom agent. It opens with three widgets: Advertised vs real APR, Score drivers, and Repayment progress. APR and score widgets read session storage written by the offer and score pages. Repayment progress reads **GET /repayment/status?applicant_id=PSC-PRIYA-001**. The question bar appends whatever was typed as a new widget title. A few phrases get a drawing: “income by”, “apr”, “score”, “repayment”, “cashflow”, and “risk”. Other titles render as an empty card. Widgets can be removed. Nothing here is saved on the server.

4. Shared interface

The look is a dark navy and charcoal base, with a light theme as well. PayPal blue is the action colour. Gold is the highlight. Risk uses green, amber, orange, and rose. Cards use a glass blur. Pages fade and slide in. Interactive cards lift on hover. The landing cards follow the pointer.

File	Role
components/ui/GlassCard.tsx	Glass card used on every screen
components/ui/Button.tsx	Primary, ghost, and gold buttons
components/ui/Workspace.tsx	Page heading, metric card, count-up, progress ring, notice, field, toast, keyboard hint
components/ui/Skeleton.tsx	Loading placeholders, including the route suspense screen
components/ui/ThemeToggle.tsx	Light and dark, stored as paypilot.theme
components/ui/SandboxBadge.tsx	Persistent sandbox disclaimer
components/ui/Confetti.tsx	Burst used when a sandbox payout completes
components/ui/SuccessAnimation.tsx	Check mark beside a landed payout
components/ui/AmbientBackground.tsx	Soft colour field behind landing and the app
components/brand/Logo.tsx	Mark and wordmark
components/landing/FloatingCards.tsx	Landing illustration
components/landing/PayPalLoginButton.tsx	PayPal-styled login control
lib/api.ts	fetch wrapper, JSON and form posts, INR formatting, persona type
lib/motion.ts	Shared fade, stagger, and hover variants
context/AuthContext.tsx	Demo session and PayPal link
context/ThemeContext.tsx	Theme preference

API surface the UI calls

vite.config.ts proxies these prefixes to 127.0.0.1:8000: /paypal, /demo, /health, /score, /income-profile, /disburse, /repayment, /offer-guard, /copilot, /explain, /counterfactuals, /affordability, and /revenue-volatility. The pages themselves call a subset of those.

Called from the UI	Used by
GET /paypal/status, GET /paypal/auth/login	Landing login
GET /paypal/public-config	One-off PayPal checkout
GET /demo/persona/priya	Overview, income grid, score, cockpit
GET /income-profile	Overview and income
POST /score, POST /counterfactuals	PayScore
GET /score/live	Top bar and repayment
POST /offer-guard, POST /offer-guard/upload	Offer guard
POST /copilot/chat	Copilot
POST /disburse, GET /disburse/{id}	Copilot approval, repayment status
POST /repayment/subscription, /order, /revenue-share	Repayment setup
POST /repayment/order/{id}/capture, GET /repayment/status	Checkout and ledger

Proxied and not called by a screen: GET /health, POST /explain, POST /affordability, and POST /revenue-volatility. Affordability and volatility are reached only when the copilot's Python agent calls them. There is no invoice screen, even though the API exposes outstanding invoices, an advance, and an invoice reminder.

5. What is planned

The plan below is the Person 3 column of the PayScore team work split (PayPal AI Hackathon, four people, submit 11 November 2026). Several week-2 screens are already in the repository. The dates are the plan, not a claim that the later weeks are unfinished in every respect.

Week 1 · 22–28 October · setup

Person 3 was asked to get the AG Studio boilerplate running and to check which MCP server tools work in the sandbox. AG Grid is running on the income feed, and a cockpit page exists. A recorded check of PayPal MCP tools is not in the frontend.

The same week, Person 1 trains the score model and ships a stub /score endpoint, Person 2 creates the sandbox app and a FastAPI skeleton, and Person 4 writes the Priya stories plus two others and a video script. The UI already assumes Priya.

Week 2 · 29 October–4 November · features

Person 3's week-2 brief is: wire the copilot to the PayPal MCP server, keep a human approval before any money moves, and build the score and offer-guard screens.

• Score and offer guard are built and already call the Python API, including SHAP factors, counterfactuals, PDF upload, and the KFS checklist.

• Human approval before money moves is built for disbursement. The chat does not create a payout until Approve is pressed, and Approve is refused until a score exists in the browser.

• Still planned: connect that copilot to the PayPal MCP server. Today the chat is a single POST /copilot/chat to the local Python agent, which picks one Python function from the wording of the question.

Week 3 · 5–11 November · polish and ship

Person 3's week-3 brief is AG Studio widgets and a custom agent. A Bryntum calendar is in scope only if the team is ahead. The current cockpit is the widget half of that brief, drawn in React. The custom agent is not started, and Bryntum is not a dependency.

Test freeze is 9–10 November. Person 4 runs the full sandbox path: login, score, offer guard, copilot, payout, EMI, and invoice paid. Invoice paid is Person 2's API and webhook step. There is no invoice screen. Persons 2 and 3 fix what breaks. The video is recorded and the entry is submitted on 11 November. Person 1 can review the script and the README remotely. Person 1 does not run the PayPal sandbox.

Still open in the UI

Item	Why it is still open	Status
PayPal MCP copilot	Week 2 plan. The chat UI can stay; the agent behind POST /copilot/chat is not the MCP server.	Planned
Custom AG Studio agent	Week 3 plan. Widgets are local cards with keyword drawings, not a Studio agent that builds views from a question.	Planned
Bryntum calendar	Week 3, and only if the score, offer, copilot, and cockpit work is already solid.	If ahead
MCP tool check	Week 1 plan. No screen or note in the frontend records which sandbox MCP tools respond.	Planned
Priya only	The written demo is Priya. Arjun and Rohan exist as API fixtures. Score, copilot, repayment, and the cockpit all call PSC-PRIYA-001.	Partial

What not to treat as future UI work

These items sit next to the frontend and are owned elsewhere, or are already decided limits:

• The score model, SHAP, counterfactuals, offer-guard maths, affordability, and the revenue-share percentage are Person 1's Python modules. The UI displays them.

• Sandbox login, Transaction Search, Payouts, Subscriptions, Orders, Invoicing, and webhooks are Person 2. The UI already calls the routes those features expose.

• The shipped RBI directory snapshot has no lender names, so the offer screen will keep saying the directory check is unverified until a retrieved list is stored. That is a data limit, not a missing component.

• OCR and the external offer-term model are off unless extra software and both server flags are set. The upload control already posts the PDF.

• Scores, affordability, and revenue share are demo estimates. The screens already say they are not a lending decision.

Suggested order from here

The screens a judge will click are already present. The plan's remaining frontend work is the agent behind the chat, then a real cockpit widget, and a calendar only if that work is already solid.

Order	Work	Touches
1	Confirm which PayPal MCP tools answer in the sandbox, and point the copilot at that server while keeping the approval card.	Copilot.tsx and the agent it calls
2	Turn the cockpit question into a real widget, backed by income, score, offer, or repayment data, instead of a title plus a keyword sketch.	Cockpit.tsx
3	Add Bryntum only after the copilot and the cockpit are demo-stable.	New dependency and a calendar view

Sources: frontend/src, frontend/package.json, frontend/vite.config.ts, frontend/README.md, docs/IMPLEMENTATION_REPORT.md, and the PayScore team work split. Where the UI README and the source disagree, this note follows the source.